Adevar Labs

Prepared for:
Degate

March 20, 2026

Solana LP Handler Audit Report

Version: after fix review

Table of Contents

1. Executive Summary

About Degate

Degate's Solana LP Handler is a wrapper around Raydium CLMM liquidity management flows. It supports position creation, liquidity increases and decreases, token swaps into the required deposit ratio, reward and fee collection, and protocol-specific security controls for integrator fee routing and operational safety.

Audit Summary

The table below summarizes identified vulnerabilities by risk level and remediation status.

Risk Level	Count	Fixed	Acknowledged
Critical	0	0	0
High	1	1	0
Medium	5	5	0
Low	4	1	3

Enhancement opportunities: 7

Overall Security Posture

The Degate team has demonstrated a strong understanding of Raydium CLMM primitives and the preliminary audit adopted a security-first mindset. However, the findings around swap planning, slippage enforcement, reward handling, and security configuration show that adversarial edge cases were not yet fully modeled. The codebase is functional, but it needs stronger defensive validation before launch.

After Fix Review Summary

The Degate team promptly resolved all High and Medium severity issues and demonstrated a strong understanding of the security implications.

Before Fix Review Summary

The audit identified 1 High severity issue related to a protocol-wide denial of service risk in **SecurityConfig**, 5 Medium severity issues affecting transfer-fee accounting, zap execution, slippage protection, initialization hardening, and price-boundary enforcement, and 4 Low severity issues centered on dust handling, fee routing, and reward accounting. Additionally, 7 enhancement opportunities were identified to improve naming clarity, validation consistency, and maintainability.

Launch Recommendations

I. Mandatory before launch:

- Resolve the 1 High severity availability issue in **SecurityConfig**, including the lamport injection denial-of-service condition and any initialization hardening required for the security configuration flow.
- Fix the 5 Medium severity issues affecting zap execution, swap slippage enforcement, transfer-fee accounting, price-boundary enforcement, and claim-path sandwich resistance before exposing user funds to production traffic.
- Re-test **swap_and_deposit**, **increase_liquidity**, and **decrease_liquidity** end-to-end against realistic Raydium CLMM pool states, including out-of-range positions, Token2022 assets, and reward-bearing pools.

II. Strongly recommended:

- Address the 4 Low severity issues around fee routing, dust handling, reward accounting, and token account canonicalization to reduce revenue leakage and unexpected user outcomes.
- Implement the 7 enhancement opportunities that improve naming clarity, native mint handling, tick-array validation consistency, and general maintainability.
- Add regression tests for cleanup swaps, reward collection, fee-owner account selection, and boundary conditions around dust and minimum swap thresholds.

III. Post-Launch Monitoring:

- Monitor failed transactions and on-chain logs for **swap_and_deposit**, **increase_liquidity**, and **decrease_liquidity** to quickly detect regressions in account routing, slippage checks, or Raydium CPI integration.
- Track fee-collection balances, reward token flows, and destination account patterns to spot fee fragmentation, missed integrator revenue, or unexpected reward routing.
- Alert on any unexpected lamport changes or reinitialization events involving the **SecurityConfig** PDA and review abnormal admin or fee-owner configuration updates promptly.

Audit Scope

- **Repository:** https://github.com/degatedev/solana_lp_handler
- **Before-Fix Review Commit Hash:** 17ea3eac2fb57ea79f69ffb9aad01627aa530134
- **After-Fix Review Commit Hash:** c09ed65b8b1ed572c0d743d69a15064134cc8fc1
- **Files/Modules in Scope:**
  - programs/*

2. Assumptions and Considerations

Centralization Risks

I. Admin authority: Controls the security config and is relied upon to conduct due diligence on the pools the protocol supports and on trusted fee manager addresses.

Trust Assumptions

II. Integrator fee enforced only on front-end: It is known that the integrator fee can be bypassed by calling the on-chain program directly. The front-end is trusted to configure a reasonable fee value.

Operational Assumptions

III. Only USDC-paired pools will be supported: The protocol will only support Raydium pools in which one of the tokens is USDC.

3. Severity Definitions

Each issue identified in this report is assigned a severity level based on two dimensions: **Impact** and **Likelihood**. These dimensions help project our team's understanding of both the potential consequences of a vulnerability and how likely a vulnerability is to be discovered and exploited in the real world.

Note: Enhancements represent non-blocking improvements--typically usability, observability, or defense-in-depth tweaks that do not pose an immediate asset risk but would improve the product's reliability and/or user experience if implemented.

Impact

Impact reflects the potential consequences of the issue--particularly on **project funds**, **user funds**, and the **availability or integrity** of the protocol.

- **High Impact**: Successful exploitation could result in a complete loss of user or protocol funds, disruption of core protocol functionality, or permanent loss of control over critical components.
- **Medium Impact**: Exploitation could cause significant disruption or partial loss of funds, but not a total compromise. May impact some users or non-core functionality.
- **Low Impact**: The issue has minor or negligible consequences. It may affect edge cases, expose metadata, or degrade performance slightly without putting funds or core logic at serious risk.

Likelihood

Likelihood reflects how easy a vulnerability is to discover and exploit by an attacker, as well as how economically attractive the exploit is to an attacker.

- **High Likelihood**: The vulnerability is trivially exploitable. This means it can be exploited by a wide range of actors without privileged access rights, with minimal capital requirements and low financial risks.
- **Medium Likelihood**: This type of vulnerability can be found and exploited with moderate effort. It might require a significant capital investment, but with manageable financial risk.
- **Low Likelihood**: Exploitation of these vulnerabilities is often technically unfeasible or requires highly specialized conditions. They may require extraordinary effort or a significant financial risk for an attacker, with a high chance of failure and minimal potential return.

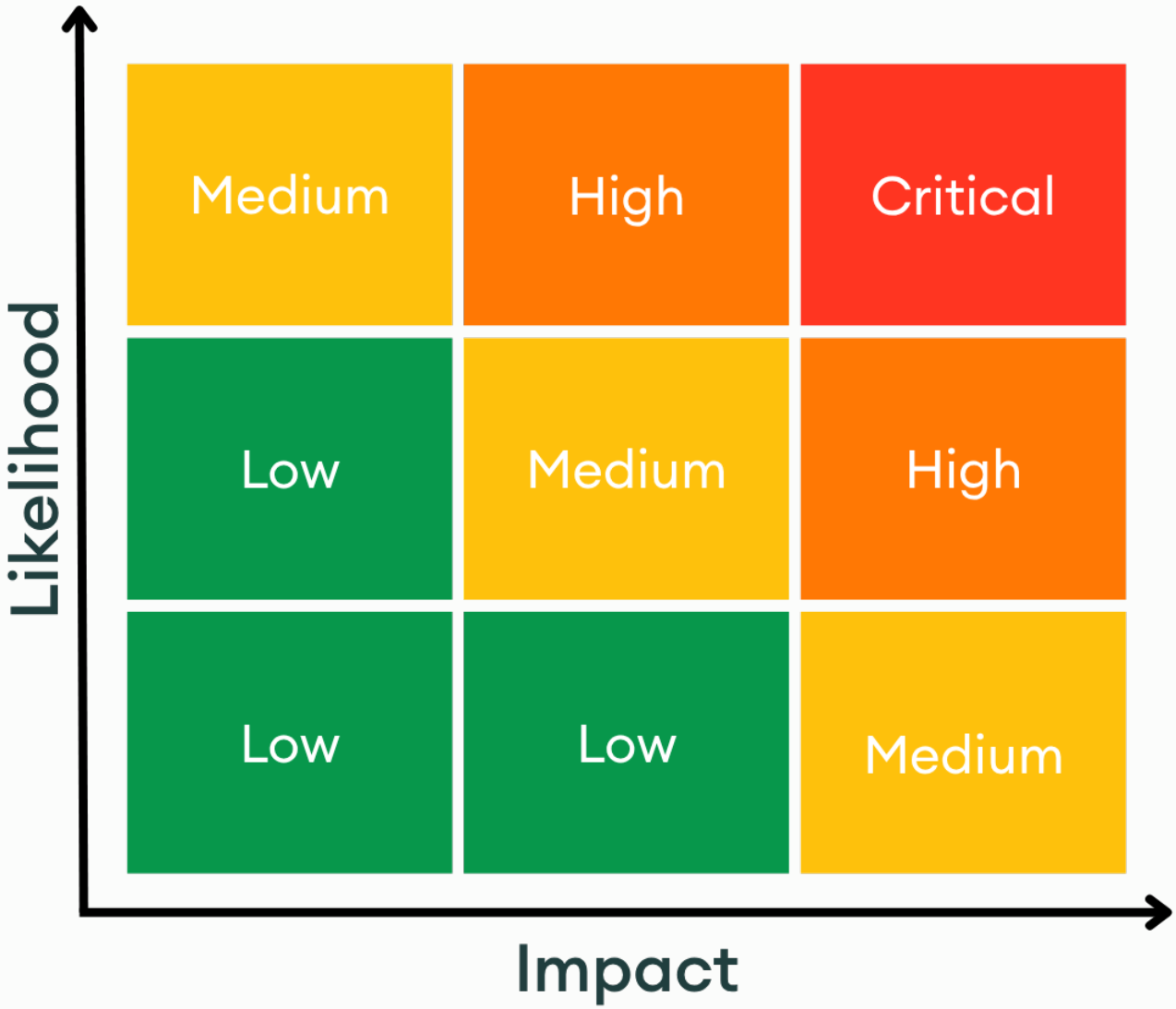
Severity Classification Matrix

By combining **Impact** and **Likelihood**, we assign a severity level using the matrix below:

- **Critical**: High impact + high likelihood (e.g. a bug that could allow anyone to drain a substantial amount of protocol funds with minimal effort)
- **High**: High impact with medium likelihood, or medium impact with high likelihood
- **Medium**: Moderate impact and/or discoverability

- **Low**: Minimal impact or unlikely to be exploited

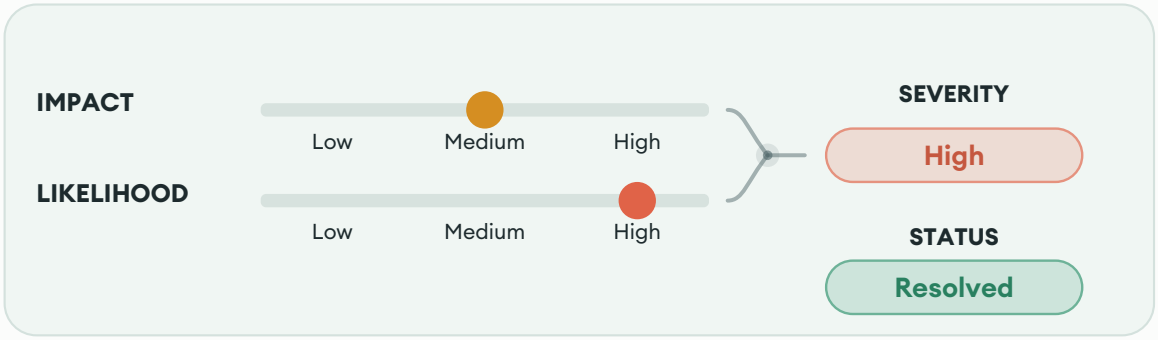
This structured approach helps teams prioritize fixes and mitigate the most dangerous threats first.



Severity Matrix

4. Findings

H01: Lamport injection into SecurityConfig PDA will DoS all protocol operations



LOCATION

solana_lp_handler/17ea3eac/programs/lp_handler/src/security/mod.rs:L508-L516

RELEVANT SNIPPET

>_ mod.rs RUST

```
506:
507: for ai in accounts.iter() {
508:     if ai.owner == &crate::ID {
509:         let min = rent.minimum_balance(ai.data_len());
510:         require!(
511:             ai.lamports() <= min,
512:             LpDepositError::SecurityProgramLamportsLeaked
513:         );
514:     }
515: }
516:
517: for b in before.signer_token_accounts.iter() {
518:     let ai = &b.account;
```

DESCRIPTION

Any external actor could block all business instructions (**swap_and_deposit**, **increase_liquidity**, **decrease_liquidity**) by sending 1 lamport to the SecurityConfig PDA. The **exit_check** in security/mod.rs validates that all program-owned accounts (**owner == crate::ID**) have lamports at most equal to **rent.minimum_balance(data_len)**; this is an absolute comparison, not a snapshot-based comparison against entry state.

```rust
for ai in accounts.iter() {
    if ai.owner == &crate::ID {
        let min = rent.minimum_balance(ai.data_len());
        require!(
            ai.lamports() <= min,
            LpDepositError::SecurityProgramLamportsLeaked
        );
    }
}
```

The SecurityConfig PDA is the only **crate::ID**-owned account in **ctx.accounts** across all business instructions. On Solana, anyone can transfer SOL to any address including program-owned PDAs. Sending even 1 lamport causes **exit_check** to fail with **SecurityProgramLamportsLeaked** for all subsequent business instructions. The attack is permissionless, essentially free (~0.000005 SOL tx fee), and repeatable after each admin remediation (close+reinit).

While no funds are at risk (transactions revert atomically, and users retain direct Raydium access), all protocol operations are blocked indefinitely. The program is deployed immutable, so the code cannot be patched.

PROOF OF CONCEPT

1. Attacker derives SecurityConfig PDA: **find_program_address(&[b"security_config"], &program;_id)**
2. SecurityConfig PDA is initialized with exactly **rent.minimum_balance(1520)** - 11,337,600 lamports
3. Attacker sends 1 lamport via **system_program::transfer(from=attacker, to=pda, amount=1)** - no special permissions needed
4. PDA now has 11,337,601 lamports
5. User calls **swap_and_deposit()**:
- **secure_entrypoint!** calls **entry_check_and_snapshot** -> passes (no lamport check at entry)
- Business logic executes: swap CPI, open_position CPI -> succeeds
- **exit_check** runs at lib.rs:54
- At security/mod.rs:509: **ai.owner == &crate;::ID** matches SecurityConfig PDA
- At security/mod.rs:510: **min = rent.minimum_balance(1520) = 11,337,600**
- At security/mod.rs:511: **ai.lamports() = 11,337,601 > min** -> **FAILS**
- Entire transaction reverts with **SecurityProgramLamportsLeaked**
6. Same failure for **increase_liquidity()** and **decrease_liquidity()**
7. Admin can recover by atomically calling **close_security_config** + **init_security_config**, but attacker can repeat for ~1 lamport + tx fee

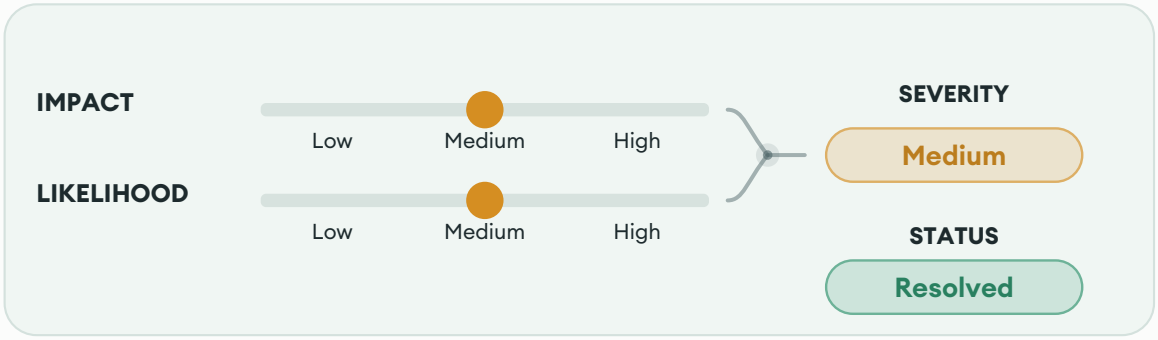
RECOMMENDATION

Change the lamport check in **exit_check** from an absolute comparison to a snapshot-based comparison: capture each program-owned account's lamport balance during **entry_check_and_snapshot**, then verify at exit that lamports have not increased beyond the entry value. This allows the check to fulfill its original purpose (preventing business logic from parking stolen SOL in program-owned accounts) while being immune to external lamport injection.

DEVELOPER RESPONSE

"The lamports check for program-owned accounts was changed from an absolute rent-based limit to an entry-snapshot/exit-relative comparison, preventing global DoS from external lamport transfers into PDAs."

M01: Token2022 transfer fees are not accounted for in lp_handler's token amount calculations



LOCATION

solana_lp_handler/17ea3eac/programs/lp_handler/src/instructions/zap_common.rs:L360-L366

RELEVANT SNIPPET

>_ zap_common.rs RUST

```rust
358: let sqrt_ratio_a_x64 = get_sqrt_price_at_tick(tick_lower_index)?;
359: let sqrt_ratio_b_x64 = get_sqrt_price_at_tick(tick_upper_index)?;
360: let computed_liquidity = liquidity_math::get_liquidity_from_amounts(
361:     sqrt_price_x64,
362:     sqrt_ratio_a_x64,
363:     sqrt_ratio_b_x64,
364:     amount_0_max,
365:     amount_1_max,
366: );
367:
368: Ok(ZapPlan {
```

DESCRIPTION

lp_handler performs all token amount calculations using raw (gross) amounts without deducting Token2022 transfer fees. This causes two distinct failures depending on the code path:

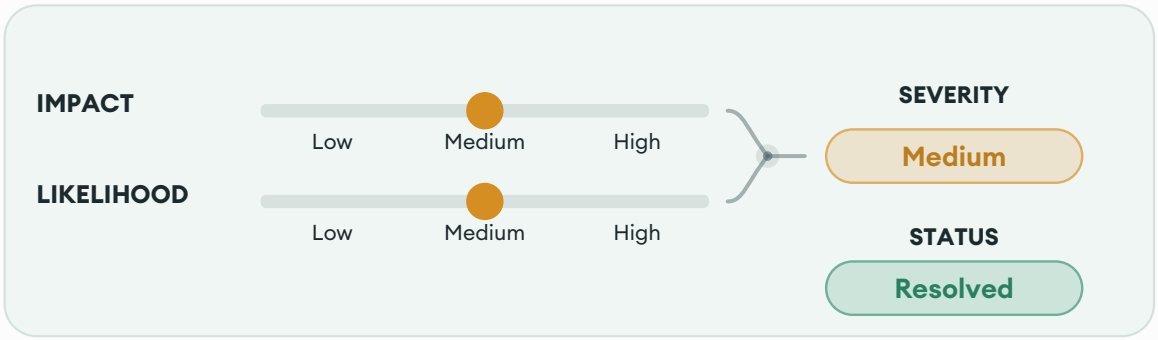
RECOMMENDATION

Account for Token2022 transfer fees throughout the program's token calculations.
- On the deposit path, mirror Raydium's approach in **open_position.rs:414-424** by deducting transfer fees before computing liquidity.
- On the decrease path, deduct transfer fees from the expected principal before comparing against the actual balance delta.

DEVELOPER RESPONSE

"Deposit/zap-side liquidity calculations are now transfer-fee-aware for Token-2022 mints, and the decrease path now reconciles expected principal against actual post-fee balance changes."

M02: prepare_zap_plan_and_swap_if_needed auto-swap on out-of-range positions overwrites the user-provided min_out enabling sandwich attacks



LOCATION

solana_lp_handler/17ea3eac/programs/lp_handler/src/instructions/zap_common.rs:L192-L221

RELEVANT SNIPPET

>_ zap_common.rs RUST

```
190:
191: let out_of_range = sqrt_price_x64_now <= sa || sqrt_price_x64_now >= sb;
192: if sqrt_price_x64_now <= sa {
193:
194:     exec_swap_input_is_token0 = false;
195:     exec_swap_amount_in = amount_1_in;
196:     exec_swap_min_out = if exec_swap_amount_in > 0 {
197:         utils::calc_min_amount_out(
198:             exec_swap_amount_in,
199:             exec_swap_input_is_token0,
200:             sqrt_price_x64_now,
201:             slippage_bps,
202:             trade_fee_rate,
203:         )?
204:     } else {
205:         0
206:     };
207: } else if sqrt_price_x64_now >= sb {
208:
209:     exec_swap_input_is_token0 = true;
210:     exec_swap_amount_in = amount_0_in;
211:     exec_swap_min_out = if exec_swap_amount_in > 0 {
212:         utils::calc_min_amount_out(
213:             exec_swap_amount_in,
214:             exec_swap_input_is_token0,
215:             sqrt_price_x64_now,
216:             slippage_bps,
217:             trade_fee_rate,
218:         )?
219:     } else {
220:         0
221:     };
222: }
223:
```

DESCRIPTION

This program offers two methods to add liquidity to a Raydium CLMM position, either through **increase_liquidity** or through **swap_and_deposit**. Both methods invoke **prepare_zap_plan_and_swap_if_needed** as a first step.

This function allows users to first swap **swap_amount_in** tokens before the next action (creating position or adding liquidity) occurs. It works as follows:
1. Checks are performed on the input parameters
2. Position ticks are compared against the existing pool spot price
2a. If out of range, the execution plan is automatically configured to swap only one of the tokens using the maximum token budget provided
2b. Otherwise the user provided plan is checked against the provided maximum token budgets
3. Execute the swap

The issue arises at step 2a where the user provided plan is overriden if the position is out of range relative to the pool spot price. **exec_swap_min_out** is what actually gets executed, overriding the user provided **swap_min_out** and is computed as follows:

```rust
exec_swap_min_out = if exec_swap_amount_in > 0 {
    utils::calc_min_amount_out(
        exec_swap_amount_in,
        exec_swap_input_is_token0,
        sqrt_price_x64_now, // manipulated price, attacker has already executed the large
  swap
        slippage_bps,
        trade_fee_rate,
    )?
}
```

The issue is that an attacker can sandwich attack this operation, executing a large swap before (front-run) the user swap and another one to take the profit after the user swap (back-run).

In **calc_min_amount_out**, the slippage parameter is applied on an already manipulated price and doesn't protect against the sandwich attack:

RUST

```rust
let amount_in = U256::from(swap_amount as u128);
let sqrt_price = U256::from(sqrt_price_x64); // already manipulated spot price
let price_q128 = (sqrt_price * sqrt_price) >> 64;
require!(!price_q128.is_zero(), LpDepositError::InvalidSqrtPrice);

let slippage_factor = U256::from(10_000u128 - slippage_bps as u128);
let fee_factor = U256::from(1_000_000u128 - trade_fee_rate as u128);

let out_before_fee_and_slippage = if input_is_token0 {
    // token0 → token1
    let raw = (amount_in * price_q128) >> 64;
    raw
} else {
    // token1 → token0
    let inv_price = ((U256::one() << 64) << 64) / price_q128;
    let raw = (amount_in * inv_price) >> 64;
    raw
};

let denom = U256::from(1_000_000u128) * U256::from(10_000u128);
let out = (out_before_fee_and_slippage * fee_factor * slippage_factor) / denom; // slippage
  applied on manipulated price
```

This issue is limited to the out-of-range auto-swap branch. The in-range path still uses the caller's swap plan and validates it only when **!out_of_range**.

RUST

```rust
if !out_of_range {
        if swap_amount_in == 0 {
            require!(swap_min_out == 0, LpDepositError::InvalidDepositAmount);
        } else if swap_input_is_token0 {
            require!(
                swap_amount_in <= amount_0_in,
                LpDepositError::InvalidDepositAmount
            );
        } else {
            require!(
                swap_amount_in <= amount_1_in,
                LpDepositError::InvalidDepositAmount
            );
        }
    }
```

PROOF OF CONCEPT

This PoC intentionally removes fees to make the mechanics easier to read. It is an illustrative economic example, not a tick-accurate Raydium CLMM replay. Exact CLMM outputs depend on active liquidity and tick crossings.

1. The user calls **swap_and_deposit** or **increase_liquidity** with:
- **amount_0_in** = 10,000
- **amount_1_in** = 0
- **slippage_bps** = 500 (5%)
- **swap_min_out** = 19000
- **trade_fee_rate** = 0 for this simplified example
- current pool spot price = 2.0
- user range upper price = 1.5

Since the current price is above the range, the sqrt_price_x64_now >= sb branch executes and the program auto-swaps all token0 into token1.

2. User's intended protection.

At the pre-attack spot price 2.0, a natural 5%-slippage lower bound is:

```rust
user_min_out = floor(10,000 * 2.0 * 0.95) = 19,000
```

Without an attack, a simple local-liquidity execution example with reserves

```rust
x = 1,000,000 token0
y = 2,000,000 token1
```

would return:

```rust
victim_out_no_attack = 2,000,000 - (2,000,000,000,000 / 1,010,000)
                     ≈ 19,801.98 token1
```

so the user's intended floor of 19,000 is reasonable.

3. Attacker front-runs in the same direction as the forced auto-swap.

The attacker sells 120,000 **token0** first. In the same fee-free illustrative model:

```rust
x1 = 1,000,000 + 120,000 = 1,120,000
y1 = 2,000,000,000,000 / 1,120,000 = 1,785,714.2857
attacker_out_token1 = 2,000,000 − 1,785,714.2857 = 214,285.7143
new_spot = y1 / x1 = 1.594387755
```

The price has moved sharply against a **token0** -> **token1** seller, but it is still above 1.5, so the same out-of-range branch still applies.

4. The program overwrites **min_out** using the manipulated spot price.

Instead of preserving 19,000, the handler recomputes:

```rust
exec_swap_min_out = floor(10,000 * 1.594387755 * 0.95)
                  = 15,146
```

5. The victim executes at a price that should have been rejected.

The victim then swaps 10,000 **token0** into the attacked pool:

```rust
x2 = 1,120,000 + 10,000 = 1,130,000
y2 = 2,000,000,000,000 / 1,130,000 = 1,769,911.5044
victim_out = y1 − y2 = 15,802.7813 token1
```

This output is:

```rust
15,802.7813 < 19,000    (fails the user's intended bound)
15,802.7813 > 15,146    (passes the overwritten on-chain bound)
```

So the transaction succeeds only because the contract replaced the user's protection with a lower, manipulable threshold.

6. The attacker back-runs.

The attacker swaps their 214,285.7143 **token1** back after the victim:

```rust
x3 = 2,000,000,000,000 / (1,769,911.5044 + 214,285.7143)
   = 1,007,964.3198

attacker_out_token0 = 1,130,000 - 1,007,964.3198
                    = 122,035.6802 token0
profit = 122,035.6802 - 120,000
       = 2,035.6802 token0
```

The attacker profits, while the victim receives materially less output than their original slippage floor should have allowed.

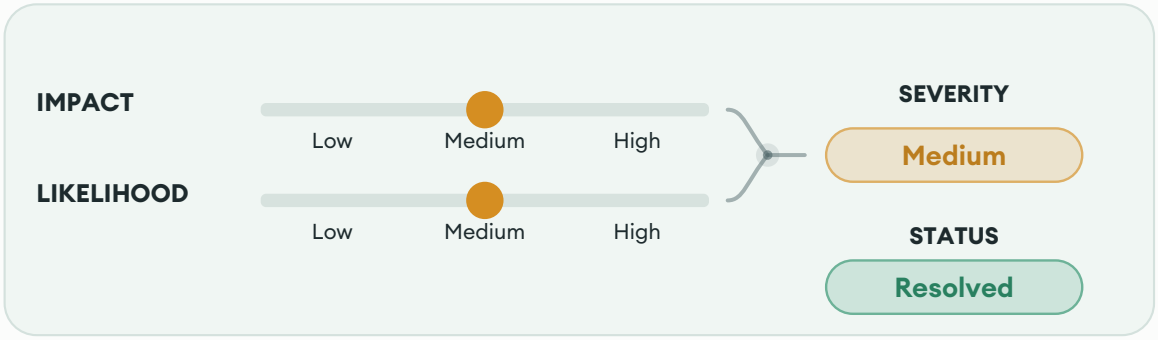
RECOMMENDATION

The core fix is to stop replacing a user-authenticated execution bound with a spot-price-derived bound. Preserve user-controlled slippage protection for the auto-swap path. The program may choose the branch on-chain, but the protection must still be user-committed.

DEVELOPER RESPONSE

"Validation for **quoted_mode** *and* **quoted_sqrt_price_x64** *was introduced to prevent out-of-range automatic swaps from overriding user-provided execution protections."*

M03: Out-of-range zap flow does not enforce price boundaries on swap, causing deposit failure or double swap fees with negligible liquidity



LOCATION

solana_lp_handler/17ea3eac/programs/lp_handler/src/instructions/zap_common.rs:L191-L222

RELEVANT SNIPPET

>_ zap_common.rs — RUST

```rust
189: let mut exec_swap_input_is_token0 = swap_input_is_token0;
190:
191: let out_of_range = sqrt_price_x64_now <= sa || sqrt_price_x64_now >= sb;
192: if sqrt_price_x64_now <= sa {
193:
194:     exec_swap_input_is_token0 = false;
195:     exec_swap_amount_in = amount_1_in;
196:     exec_swap_min_out = if exec_swap_amount_in > 0 {
197:         utils::calc_min_amount_out(
198:             exec_swap_amount_in,
199:             exec_swap_input_is_token0,
200:             sqrt_price_x64_now,
201:             slippage_bps,
202:             trade_fee_rate,
203:         )?
204:     } else {
205:         0
206:     };
207: } else if sqrt_price_x64_now >= sb {
208:
209:     exec_swap_input_is_token0 = true;
210:     exec_swap_amount_in = amount_0_in;
211:     exec_swap_min_out = if exec_swap_amount_in > 0 {
212:         utils::calc_min_amount_out(
213:             exec_swap_amount_in,
214:             exec_swap_input_is_token0,
215:             sqrt_price_x64_now,
216:             slippage_bps,
217:             trade_fee_rate,
218:         )?
219:     } else {
220:         0
221:     };
222: }
223:
224: let balance_0_before = accounts.signer_token0_account().amount;
```

DESCRIPTION

The zap flow for out-of-range deposits swaps the user's entire balance of the opposing token without enforcing that the swap cannot push the pool price into the position's tick boundary. Since the swap direction inherently moves the price toward the range, a large enough swap will cross the boundary, producing a degenerate deposit.

Root cause:

The out-of-range override makes a decision at time T (position is out of range, swap everything) but never validates that the decision's precondition still holds at time T+1 (after the swap):

1. The program checks **sqrt_price_x64_now >= sb** (or **<= sa**) and concludes only one token is needed (line 191)
2. It overrides the caller's swap parameters, setting **exec_swap_amount_in** to the user's full opposing-token balance (lines 195, 210)
3. The swap executes via **swap_v2_common** with **sqrt_price_limit_x64 = 0** -- no price bound (line 275)
4. After the swap, the program reloads balances and computes **amount_0_max** / **amount_1_max** (lines 319-335) but does not re-check whether the position is still out of range
5. It computes **get_liquidity_from_amounts** at the new (post-swap) pool price (lines 354-366) and proceeds to deposit -- even if the price has crossed the tick boundary

RUST

```
swap_v2_common(
    accounts,
    exec_swap_amount_in,    // full balance of opposing token
    exec_swap_min_out,
    0,                       // sqrt_price_limit_x64 = 0 (no limit)
    exec_swap_input_is_token0,
    swap_remaining_slice.to_vec(),
)?;
```

When **sqrt_price_limit_x64 = 0**, Raydium defaults to **MIN_SQRT_PRICE + 1** or **MAX_SQRT_PRICE - 1** (swap_v2.rs:153-158) -- effectively no boundary. The swap runs until the full input is consumed, regardless of how far the price moves.

How the swap direction guarantees boundary crossing for large amounts:

When a position is out of range, only one token is needed. The override swaps the opposing token into the needed one:

- Price below **tick_lower** (only token0 needed): swaps token1 -> token0, which pushes price UP -- toward the range
- Price above **tick_upper** (only token1 needed): swaps token0 -> token1, which pushes price DOWN -- toward the range

The swap always moves price toward the position's tick boundary. For small amounts, the price stays out of range and the deposit works normally. For amounts large enough relative to pool liquidity, the price crosses into (or through) the range. The off-chain solver (tests/utils.ts **solveZapTwoSidedCLMM**) also swaps the full amount of the opposing token for out-of-range positions without checking for overshoot, confirming this is the designed flow, not an edge case.

Why calc_min_amount_out **is insufficient:**

The out-of-range override computes **exec_swap_min_out** via **calc_min_amount_out** (lines 196-206), which uses **amount_in × spot_price × (1 - fee) × (1 - slippage_bps)**. This effectively caps the swap's total price impact to approximately **slippage_bps**. For very large overshoots where total impact exceeds **slippage_bps**, the min_out check fails and the transaction reverts -- providing incidental protection.

However, **calc_min_amount_out** is a total impact cap, not a directional price boundary. Boundary crossing passes the check whenever:

boundary_gap < slippage_bps

where **boundary_gap = |current_price - tick_boundary| / current_price**. In the PoC below, the gap is 0.33%. Users on moderate-TVL pools realistically set **slippage_bps** to 100-200 bps to avoid reverts from normal price impact. Whenever the boundary gap falls within this slippage tolerance, **calc_min_amount_out** cannot prevent the swap from crossing the tick boundary. At 100 bps slippage, deposits up to 75 SOL ($11,288) overshoot and pass the check. At 200 bps, deposits up to 100+ SOL ($15,050+) pass.

This is fundamentally the wrong constraint for this problem. **calc_min_amount_out** limits *how far* the price can move in total. What is needed is a limit on *where* the price can move to -- a directional boundary that stops the swap at the tick, which is exactly what **sqrt_price_limit_x64** provides.

Violation of user intent:

The user explicitly selected an out-of-range position -- a deliberate strategy (e.g., limit-order-like entries, accumulating a single token at a target price). The program detects this intent correctly at line 191: **sqrt_price_x64_now <= sa || sqrt_price_x64_now >= sb** triggers the single-sided override. But when the swap overshoots, the position silently becomes in-range -- a fundamentally different position type that requires both tokens and produces different risk/return characteristics. The user chose a single-sided position but receives a near-zero-liquidity two-sided position with double swap fees -- an outcome they never intended and had no way to prevent on-chain.

Post-swap consequences:

After overshoot, the user has dust of the swapped-away token (from integer rounding in tick-crossing math) and a large balance of the target token. The post-swap price is now in-range, requiring both tokens for liquidity provision. Two outcomes follow:

Path A (revert): For very large deposits where total price impact exceeds **slippage_bps**, **calc_min_amount_out** catches the overshoot and reverts. No fund loss, but the deposit is blocked entirely -- even though a partial swap up to the boundary would have succeeded with **sqrt_price_limit**.

Path B (dust leftover): For deposits where total impact stays within **slippage_bps** (the common case at realistic slippage settings), the swap passes the min_out check. **get_liquidity_from_amounts** returns **min(tiny, huge) = tiny**. Raydium adds negligible liquidity (fractions of a cent). Nearly the entire deposit becomes leftover. If **return_mint** is set, **swap_back_remaining_and_emit_increase_event** (line 383) swaps the massive leftover back in the opposite direction. The user pays swap fees twice on nearly their full deposit for a worthless position.

PROOF OF CONCEPT

This PoC demonstrates Path B -- the swap overshoots the tick boundary, passes the **calc_min_amount_out** check, and results in negligible liquidity with double swap fees.

Scenario: Narrow range near current price

Pool: SOL/USDC, current price = $150.50. User wants to open a position with range [$148, $150] -- a below-market range. When out of range (price above upper tick), only token1 (USDC) is needed. Pool has liquidity L = 100,000 in the active tick range. Raydium fee = 25 bps. Slippage = 100 bps (realistic for moderate-TVL pool).

Boundary gap: ($150.50 - $150) / $150.50 = 0.33%. Since 0.33% < 1.0% (slippage), the **calc_min_amount_out** check cannot prevent boundary crossing.

Key formulas:
- Token0 swap (Raydium **sqrt_price_math.rs:16-28**): **$\sqrt{P'} = \sqrt{P} \times L / (L + \Delta x \times \sqrt{P})$**. Inverted for threshold: **$\Delta x = L \times (1/\sqrt{P'} - 1/\sqrt{P})$**, gross input: **$\Delta x / (1 - fee)$**. Swap output: **$out = L \times (\sqrt{P} - \sqrt{P'})$**.
- **calc_min_amount_out** (lp_handler **utils.rs:36**): **amount_in × price × (1 - fee) × (1 - slippage)**

Derived values for this scenario:
- SOL to reach boundary: **100,000 × (1/√150 - 1/√150.5) / 0.9975 = 13.6 SOL** ($2,048)
- Swap 50 SOL output: **$\sqrt{P'} = 100{,}000 \times \sqrt{150.5} / (100{,}000 + 49.875 \times \sqrt{150.5}) = 12.194$**, giving **out = 100,000 × (12.268 - 12.194) = $7,461 USDC**. Post-swap price: **$12.194^2 = \$148.68$** -- now in-range.
- **calc_min_amount_out(50, $150.50, 25bps, 100bps) = 50 × 150.5 × 0.9975 × 0.99 = $7,431**. Since $7,461 > $7,431, **the check passes**.
- Swap fee per leg: **50 × 150.5 × 0.0025 = $18.81** (leg 1), **7,461 × 0.0025 = $18.65** (leg 2). Total: **$37.46**.

1. User holds SOL. Calls **swap_and_deposit(amount_0_in = 50 SOL, amount_1_in = 0, return_mint = SOL_MINT, slippage_bps = 100, ...)**. Total deposit value: $7,525.
2. **sqrt_price_x64_now > sb** -- price $150.50 is above the range. Auto-swap fires (line 207): sells 50 SOL for USDC, pushing price DOWN. **sqrt_price_limit_x64 = 0**.
3. Only 13.6 SOL is needed to reach the boundary at $150. The remaining 36.4 SOL pushes price to $148.68 -- past the boundary into the range. The min_out check passes (derived above).
4. Post-swap: **amount_0_max** = dust (a few lamports from integer rounding), **amount_1_max** = $7,461 USDC. Price is now in-range, so both tokens are required.
5. **get_liquidity_from_amounts(dust, $7,461)** returns **min(tiny, huge) = tiny**. Raydium CPI adds negligible liquidity. Position value: ~$0.
6. Leftover: ~$7,461 USDC. Since **return_mint = SOL_MINT**, **swap_back_remaining_and_emit_increase_event** swaps it all back to SOL. Second swap fee: $18.65.
7. Final state: user loses $37.46 in total swap fees. Position holds near-zero liquidity. The user round-tripped $7,461 through two swaps for nothing.

RECOMMENDATION

Use **sqrt_price_limit_x64** in the Raydium swap CPI to enforce the tick boundary as a price floor/ceiling:

```rust
let sqrt_price_limit = if exec_swap_input_is_token0 {
    // Selling token0, price goes DOWN -- stop at upper tick
    get_sqrt_price_at_tick(tick_upper_index)?
} else {
    // Selling token1, price goes UP -- stop at lower tick
    get_sqrt_price_at_tick(tick_lower_index)?
};

swap_v2_common(
    accounts,
    exec_swap_amount_in,
    exec_swap_min_out,
    sqrt_price_limit,        // enforce tick boundary
    exec_swap_input_is_token0,
    swap_remaining_slice.to_vec(),
)?;
```

Raydium natively handles partial fills when the price limit is reached -- the swap stops at the boundary and the unconsumed input remains in the user's account. This ensures:

1. The post-swap price stays at or beyond the tick boundary, preserving the single-sided deposit invariant
2. The price limit acts as an on-chain price guarantee, providing sandwich protection without relying on **calc_min_amount_out**
3. Unconsumed input flows naturally into **swap_back_remaining_and_emit_increase_event** as leftover

As an additional safeguard, validate after any swap that the position's range status has not changed:

```rust
let sqrt_price_x64_after = {
    let pool_state = accounts.pool_state().load()?;
    pool_state.sqrt_price_x64
};
let out_of_range_after = sqrt_price_x64_after <= sa || sqrt_price_x64_after >= sb;
require!(
    out_of_range == out_of_range_after,
    LpDepositError::PriceBoundaryCrossed
);
```

This provides defense in depth for both paths: out-of-range swaps cannot push the price into the range, and in-range swaps cannot push the price out of the range. The **sqrt_price_limit** prevents overshoot under normal conditions, while the post-swap check catches edge cases where external state changes between the swap CPI and the reload.

DEVELOPER RESPONSE

"The main zap swap now enforces **quoted_mode**, **quoted_sqrt_price_x64**, **swap_min_out**, *and* **sqrt_price_limit_x64** *tick-boundary checks together, preventing automatic swaps from continuing past the position boundary."*

M04: Permisionless SecurityConfig initialization can be front-run

LOCATION

solana_lp_handler/17ea3eac/programs/lp_handler/src/instructions/security_config.rs:L81

RELEVANT SNIPPET

>_ security_config.rs RUST

```
79: pub struct InitSecurityConfig<'info> {
80:     #[account(mut)]
81:     pub authority: Signer<'info>,
82:
83:     #[account(
```

DESCRIPTION

The **security_config** PDA is initialized by **init_security_config**, which accepts any signer and sets that signer as **security_config.authority**. Because all later **update_security_config** and **close_security_config** checks trust the stored authority, the first account to initialize this PDA permanently controls the pool whitelist and fee-owner whitelist.

This is not only a deployment-time race. The same takeover window reopens whenever **close_security_config** is called: closing the mandatory config PDA disables protected flows until the PDA is created again, and since re-initialization is permissionless, any external actor can recreate it first and seize authority. An attacker who wins the initialization or re-initialization race can deny service by configuring unusable security policy, effectively rendering the protocol unusable. However, the re-initialization risk is low and is effectively an admin error due to the ability to close and re-open the PDA atomically within the same transaction.

Important note: The previous audit finding, LPH-011, mentions that the deployment of the program will be made atomically in the same tx as **init_security_config**. However, the current Solana deployment model doesn't support this based on the official docs. There exists a visibility delay which means that programs deployed at slot N only become effective at slot N+1. Any attempt to invoke the program before slot N+1 results in a **DelayVisibility** error.

RECOMMENDATION

Restrict **init_security_config** to a trusted governance or admin authority instead of allowing any signer.

DEVELOPER RESPONSE

*"**init_security_config** still requires a restricted bootstrap admin, but that admin is no longer hardcoded in the repository. The deploy script injects the current deployer wallet via the **SECURITY_ADMIN** environment variable before **anchor build**, preserving restricted initialization while fitting the existing deployment flow."*

M05: Claiming path (liquidity=0) in decrease_liquidity fails to protect against sandwich attacks due to computing amount-out threshold using on-chain spot price

LOCATION

solana_lp_handler/17ea3eac/programs/lp_handler/src/instructions/decrease_liquidity.rs:L384-L395

RELEVANT SNIPPET

>_ decrease_liquidity.rs — RUST

```
382: let mut total_out_in_target: u64 = 0;
383: if total_other_in > 0 {
384:     // 为了避免"用旧价格估 min_out"导致过严/过松，在 CPI swap_v2 前重新读取 pool_state 的最新价
     格。
385:     let sqrt_price_x64_for_min_out = {
386:         let pool_state = ctx.accounts.pool_state.load()?;
387:         pool_state.sqrt_price_x64
388:     };
389:     let swap_other_amount_threshold = utils::calc_min_amount_out(
390:         total_other_in,
391:         input_is_token0,
392:         sqrt_price_x64_for_min_out,
393:         slippage_bps,
394:         ctx.accounts.amm_config.trade_fee_rate,
395:     )?;
396:
397:     // dust 处理：
```

DESCRIPTION

When **decrease_liquidity** is called with **convert_to_usdc = true**, the non-target token is swapped into the target token. The swap's minimum output threshold (**swap_other_amount_threshold**) is computed entirely on-chain using the current pool price:

```rust
let sqrt_price_x64_for_min_out = {
    let pool_state = ctx.accounts.pool_state.load()?;
    pool_state.sqrt_price_x64                          // current on-chain price
};
let swap_other_amount_threshold = utils::calc_min_amount_out(
    total_other_in,
    input_is_token0,
    sqrt_price_x64_for_min_out,                        // manipulable
    slippage_bps,
    ctx.accounts.amm_config.trade_fee_rate,
)?;
```

calc_min_amount_out computes **amount × price × (1 - fee) × (1 - slippage)** using the pool's spot price. Since both the min_out threshold and the swap execute against the **same pool state within the same transaction**, the threshold always passes as long as the swap's own price impact stays within **slippage_bps**. The **slippage_bps** parameter does not protect against external price manipulation that occurred **before** the transaction.

A sandwich attacker can manipulate the pool price before the victim's transaction (e.g., via Jito bundle ordering). The victim's decrease and convert swap both execute at the manipulated price. **calc_min_amount_out** computes the threshold from the same manipulated price, so the check passes trivially. The attacker then reverses the manipulation in a follow-up transaction, profiting from the round trip.

This contrasts with the deposit path (**swap_and_deposit**), where the user supplies **swap_min_out** computed off-chain from the market price, providing effective sandwich protection because the threshold is anchored to a pre-manipulation price. The **decrease_liquidity** instruction offers no equivalent parameter for the convert swap.

For full withdrawals (**liquidity > 0**), the impact is partially mitigated: **mint_amount_0** and **mint_amount_1** provide user-supplied slippage protection for the liquidity withdrawal step (Raydium checks **decrease_amount >= mint_amount**). If the attacker manipulates price enough to profit on the convert swap, the withdrawal amounts also shift, and properly set **mint_amount_0/1** values may catch the manipulation and revert the transaction before the convert swap executes. This narrows the attacker's profitable window.

However, for **claim-only calls** (**liquidity = 0** with **convert_to_usdc = true**), Raydium skips the **mint_amount_0/1** check entirely (Raydium decrease_liquidity.rs:221: **if liquidity > 0**). The only remaining parameter is **slippage_bps**, but as described above, it computes the threshold from the same manipulated on-chain price, so it provides no actual sandwich resistance. This leaves the claim+convert path with zero effective protection: accumulated trading fees and rewards are claimed and converted at whatever price the attacker sets.

PROOF OF CONCEPT

Claim+convert scenario (no mint_amount **protection):**

1. Pool: SOL/USDC, fair price = $100. User has accumulated 25 SOL in token0 trading fees and 2500 USDC in token1 trading fees.
2. User calls **decrease_liquidity(0, 0, 0, USDC_MINT, 50, 1200, true)** - claim only, convert all to USDC, 50 bps slippage.

3. **liquidity = 0** -> Raydium skips **mint_amount_0/1** check. Fees are claimed at whatever the current price is.
4. **Attacker front-run**: sells SOL into the pool, pushing price from $100 to $97.
5. **Convert swap**: **total_other_in = 25 SOL** (the non-USDC token0 fees). **calc_min_amount_out(25, true, sqrt($97), 50, fee)** -> threshold ≈ $2407 (computed from manipulated $97 price).
6. Swap executes at ~$97: user gets ~$2412 for their 25 SOL (passes the $2407 threshold).
7. **Attacker back-run**: buys back SOL at ~$97, profiting from the round trip. User received ~$2412 instead of ~$2500 at fair price. **User loss: ~$88** from the convert swap.
8. The user's **slippage_bps = 50** provided no protection: **calc_min_amount_out** computed the threshold from the $97 manipulated price, not from the $100 fair market price.

RECOMMENDATION

Add an optional user-supplied **convert_min_out** parameter to **decrease_liquidity** that, when non-zero, overrides the on-chain computed **swap_other_amount_threshold**. This lets the frontend compute the minimum acceptable output from the trusted market price off-chain, anchoring the slippage check to a pre-manipulation reference:

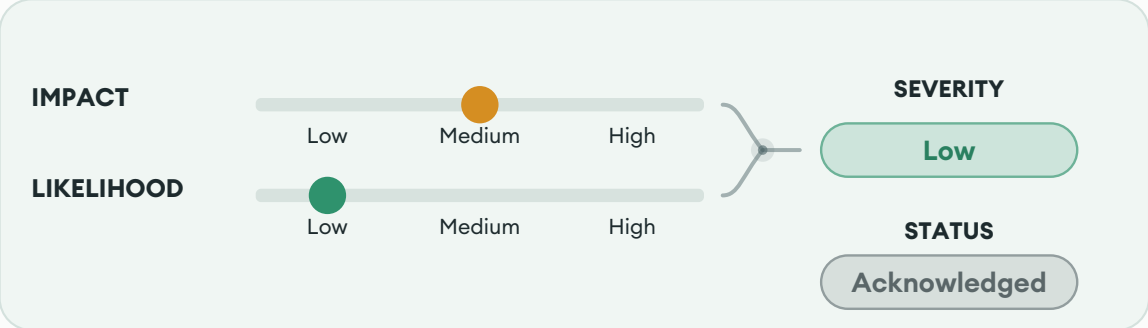
```rust
pub fn decrease_liquidity(
    liquidity: u128,
    mint_amount_0: u64,
    mint_amount_1: u64,
    swap_to_token_mint: Pubkey,
    slippage_bps: u16,
    fee_percent: u16,
    convert_to_usdc: bool,
    convert_min_out: u64,    // NEW: user-supplied floor for convert swap output
) -> Result<()> {
    // ...
    let swap_other_amount_threshold = if convert_min_out > 0 {
        convert_min_out
    } else {
        utils::calc_min_amount_out(total_other_in, ..., slippage_bps, ...)?
    };
```

This mirrors the deposit path's design where **swap_min_out** is computed off-chain and supplied by the user. This is especially critical for claim+convert operations where **mint_amount_0/1** provide no protection.

DEVELOPER RESPONSE

"Quote constraints and price protections were added to the claim/convert path, aligning it with the deposit path."

L01: Reused swap_remaining accounts can cause swap_and_deposit / increase_liquidity to revert



LOCATION

solana_lp_handler/17ea3eac/programs/lp_handler/src/instructions/swap_and_deposit.rs:L244

RELEVANT SNIPPET

>_ swap_and_deposit.rs — RUST

```
242:     plan.amount_0_max,
243:     plan.amount_1_max,
244:     plan.swap_remaining,
245:     position_nft_mint,
246: )?;
```

DESCRIPTION

Both entrypoints split **ctx.remaining_accounts** once, save a single **swap_remaining** slice, and later reuse that exact slice for a second Raydium **swap_v2** during leftover consolidation.

The split happens in **zap_common.rs**:

RUST

```
let sep = crate::ID;
let sep_index = remaining_accounts
    .iter()
    .position(|a| a.key() == sep)
    .ok_or(LpDepositError::InvalidRemainingAccounts)?;
let (swap_remaining_slice, rest) = remaining_accounts.split_at(sep_index);
let action_remaining_slice = &rest[1..]; // Skip the delimiter itself
```

The saved slice is returned in **ZapPlan** and used in the call to **swap_back_remaining_and_emit_increase_event**. That reuse is unsafe because Raydium **swap_v2** does not treat **remaining_accounts** as generic extras. They are the path-specific CLMM state for one swap from one starting pool state. Raydium parses those accounts as:
- optional **TickArrayBitmapExtension**
- ordered **TickArrayState** accounts

In **swap_v2.rs**, the swap direction is derived from the chosen input/output vaults then asks the pool for the first initialized tick array for that direction. Swap execution then requires the supplied array sequence to match the direction-specific path.

```rust
// swap.rs
let (mut is_match_pool_current_tick_array, first_valid_tick_array_start_index) =
    pool_state.get_first_initialized_tick_array(&tickarray_bitmap_extension, zero_for_one)?;
let mut current_valid_tick_array_start_index = first_valid_tick_array_start_index;

let mut tick_array_current = tick_array_states.pop_front().unwrap();
// find the first active tick array account
for _ in 0..tick_array_states.len() {
    if tick_array_current.start_tick_index == current_valid_tick_array_start_index {
        break;
    }
    tick_array_current = tick_array_states
        .pop_front()
        .ok_or(ErrorCode::NotEnoughTickArrayAccount)?;
}
```

This can revert in two distinct ways:

1. Opposite-direction revert
The main swap and cleanup swap can run in opposite directions. In that case the cleanup swap may need a different first tick array than the one quoted for the main swap. Raydium's own unit test shows this explicitly: for the same pool state, **get_first_initialized_tick_array(true)** returns a different result than **get_first_initialized_tick_array(false)**. Source here.

2. Same-direction revert
Even if both swaps go in the same direction, the second swap starts from a mutated pool state. The main swap updates pool_state.tick_current, pool_state.sqrt_price_x64, and active liquidity. The liquidity action then can update **pool_state.liquidity** if the current tick lies inside the position range. As a result, the cleanup swap may require a different leading array or one additional array beyond the original quote. If the reused list does not cover the new path, Raydium fails when it advances to the next required tick array here:

```rust
// swap.rs
if !next_initialized_tick.is_initialized() {
    let next_initialized_tickarray_index = pool_state
        .next_initialized_tick_array_start_index(
            &tickarray_bitmap_extension,
            current_valid_tick_array_start_index,
            zero_for_one,
        )?;
    if next_initialized_tickarray_index.is_none() {
        return err!(ErrorCode::LiquidityInsufficient);
    }
...
```

A user can submit a transaction with a valid initial swap quote, execute the first half of the zap correctly, and still have the entire instruction revert only because the optional cleanup swap reused a stale account bundle.

RECOMMENDATION

Do not reuse **plan.swap_remaining** for the cleanup swap.

The clean fix is to model the cleanup swap as a distinct Raydium swap with its own account slice:

- **main_swap_remaining**
- **action_remaining**
- **swap_back_remaining**

The program should use those three slices independently. If an atomic cleanup swap is retained, the client must compute **swap_back_remaining** against the predicted post-main-swap and post-action state, because the action can also modify **pool_state.liquidity**.

DEVELOPER RESPONSE

"Failure is possible only in a rare edge case where the main swap stops exactly at a tick-array boundary and cleanup must cross into the next tick array. In that case, the transaction reverts, user funds remain unaffected, and a retry succeeds. We assess the probability as very low, with no fund-safety risk."

L02: Dust reward branch confiscates 100% of the non-target reward and omits it from fee accounting

LOCATION

solana_lp_handler/17ea3eac/programs/lp_handler/src/instructions/decrease_liquidity.rs:L408-L412

RELEVANT SNIPPET

>_ decrease_liquidity.rs — RUST

```
406: };
407:
408: if principal_other_in == 0 && min < crate::consts::MIN_USDC_SWAP_AMOUNT {
409:     msg!(
410:         "skip reward swap (claim-only dust): below MIN_USDC_SWAP_AMOUNT, transfer inpu
       t to fee"
411:     );
412:     zap_common::transfer_fee(
413:         &ctx.accounts.signer,
414:         &ctx.accounts.fee_owner,
```

DESCRIPTION

In the **convert_to_usdc** flow, the dust-handling branch at **decrease_liquidity** triggers when **principal_other_in == 0** and the computed swap amount is below **MIN_USDC_SWAP_AMOUNT**.

Because this branch only executes when **principal_other_in == 0**, **total_other_in** is effectively equal to the entire reward on that side. As a result, the full reward is taken as a fee, regardless of **fee_percent**.

The issue is also invisible to downstream accounting. Later fee calculation only records **integrator_fee_target** on the target-mint side at **decrease_liquidity.rs**.

RUST

```
let integrator_fee_target = reward_total_in_target
    .checked_mul(fee_percent as u64)
    .ok_or(LpDepositError::MathOverflow)?
    .checked_div(10_000)
    .ok_or(LpDepositError::MathOverflow)?;
```

and the emitted **LpHandlerDecreaseLiquidityEvent** does not reflect the confiscated non-target-side amount.

RECOMMENDATION

Replace the dust branch with behavior that preserves the configured fee model:
- Charge only **reward_other_in * fee_percent / 10_000** and return the remainder.
- Record and emit any fee taken from the non-target side so **LpHandlerDecreaseLiquidityEvent** reflects the actual fee collected.

DEVELOPER RESPONSE

"This branch is an intentional product-design choice: when estimated output into the target token is too small and may result in a zero-output or economically meaningless swap, the reward input is transferred directly to the fee address. We intentionally do not change this branch to a partial **fee_percent** *charge because that would alter product semantics. The residual risk is known and accepted."*

L03: Unconstrained fee token accounts will lead to fee fragmentation

LOCATION

solana_lp_handler/17ea3eac/programs/lp_handler/src/instructions/decrease_liquidity.rs:L82-L93

RELEVANT SNIPPET

>_ decrease_liquidity.rs RUST

```rust
pub fee_owner: SystemAccount<'info>,

#[account(mut,
    token::mint = token_vault_0.mint,
    token::authority = fee_owner,
)]
pub fee_token0_account: Box<InterfaceAccount<'info, TokenAccount>>,

#[account(
    mut,
    token::mint = token_vault_1.mint,
    token::authority = fee_owner,
)]
pub fee_token1_account: Box<InterfaceAccount<'info, TokenAccount>>,

/// AMM 配置账户（swap 和 position 都需要通过 pool_state 关联）
```

DESCRIPTION

The **decrease_liquidity** account model does not enforce that **fee_token0_account** and **fee_token1_account** are the canonical associated token accounts for fee_owner. In **decrease_liquidity.rs** both fee token accounts are only constrained by **token::mint = token_vault_*.mint** and **token::authority = fee_owner**. That means any token account for the correct mint whose authority is the whitelisted **fee_owner** will pass.

The shared security wrapper validates only that **fee_owner** itself is on the **security_config.fee_owners** whitelist. The codebase defines **InvalidFeeTokenAccount** in errors.rs, but there is no current validation path that derives the expected fee ATA and compares it to the supplied account.

This contradicts the documented behavior. The README says the program "validates the fee_owner whitelist entry and fee ATA accounts" and more specifically claims fee_token0_account / fee_token1_account "must be the ATA for fee_owner and the corresponding mint".

RECOMMENDATION

If the intended invariant is a canonical fee vault per mint, derive the expected ATA on-chain for (**fee_owner**, **vault_0_mint**) and (**fee_owner**, **vault_1_mint**) using the correct token program ID for each mint, and reject mismatches with **InvalidFeeTokenAccount**.

If arbitrary fee_owner-controlled token accounts are actually intended, then the current README and account comments should be corrected to remove the ATA guarantee, and the dead **InvalidFeeTokenAccount** error path should be removed or implemented explicitly.

DEVELOPER RESPONSE

*"**fee_token0_account** and **fee_token1_account** are now required to be the canonical ATA for the corresponding mint owned by **fee_owner**, preventing fee fragmentation."*

L04: Lack of explicit handling of external farming rewards in decrease_liquidity will cause loss of fee revenue and uncontrolled reward routing

LOCATION

solana_lp_handler/17ea3eac/programs/lp_handler/src/instructions/decrease_liquidity.rs:L289-L292

RELEVANT SNIPPET

>_ decrease_liquidity.rs — RUST

```
287:
288: // 所有情况：如果本次操作没有带来任何余额变化（两边增量都为 0），直接失败
289: require_log!(
290:     signer_token0_amount > 0 || signer_token1_amount > 0,
291:     LpDepositError::NoBalanceChange
292: );
293:
294: // LPH-004: 避免用 saturating_sub 静默吞掉 "actual < principal" 的边界状态。
```

DESCRIPTION

The **decrease_liquidity** instruction charges an integrator fee (**fee_percent**) on trading fee rewards collected from Raydium positions. However, Raydium CLMM pools can also distribute up to 3 external farming reward tokens (governance tokens, incentive programs, etc.), and the instruction has no explicit logic to handle them. This creates two problems: fee owners lose revenue, and reward routing lacks granular control.

Why farming reward accounts are required: Raydium's **decrease_liquidity_v2** enforces strict account count validation when **CollectReward** is enabled - if reward accounts are missing from **remaining_accounts**, the CPI fails with **InvalidRewardInputAccountNumber**, blocking ALL operations on that pool. The frontend must include them for the instruction to work at all.

No fee collection on farming rewards: After the CPI, the instruction only measures balance deltas of **signer_token0_account** and **signer_token1_account** (lines 281-286) and computes integrator fees exclusively on those deltas (lines 298-321, 489-497, 589-598). Farming reward tokens - transferred to separate accounts via **remaining_accounts** - are never tracked and never fee-charged. In pools with active farming incentive programs, farming rewards can exceed trading fee APR, meaning the majority of reward value bypasses the fee mechanism entirely.

Uncontrolled reward routing: Because the instruction doesn't explicitly manage farming rewards, where they end up is entirely determined by which accounts the transaction builder places in **remaining_accounts**. The security layer (security/mod.rs lines 398-434) permits token accounts owned by **signer**, **recipient**, or whitelisted **fee_owner**. This means:

- The backend (which constructs transactions per the LPH-019 trust model) could route farming rewards to fee_owner's ATAs, capturing 100% of farming rewards - far exceeding the **fee_percent** cap enforced on trading fees.
- Conversely, rewards could be routed to the signer, bypassing the recipient entirely when **signer != recipient**.

The contract enforces **fee_percent** as a precise cap on trading fee revenue, but has zero control over farming reward distribution. The same **fee_percent = 1200** (12%) that limits trading fee extraction doesn't apply to farming rewards at all - they can be taken in full or left entirely, depending on how **remaining_accounts** is constructed.

Rewards-only claims revert: When a position has accumulated farming rewards but zero trading fees (e.g., out-of-range position), the **NoBalanceChange** check (lines 288-293) reverts the entire transaction, rolling back farming reward transfers and preventing users from claiming through **lp_handler**.

Overall, fee owners miss their share of farming reward value when users claim through lp_handler. However, funds are never locked or lost - users can always claim farming rewards by interacting with Raydium directly, bypassing lp_handler entirely. The uncontrolled routing is similarly limited in impact since the same direct Raydium interaction is always available as an alternative.

PROOF OF CONCEPT

Scenario 1 - Fee revenue loss:

1. Pool: SOL/USDC with an active farming reward program distributing token X at 20% APR
2. User has a position with $10K liquidity, accumulated 50 USDC in trading fees and 200 token X (~$100) in farming rewards
3. User calls **decrease_liquidity(0, 0, 0, USDC_MINT, 50, 1200, true)** - claim with 12% integrator fee
4. Frontend includes signer's reward ATAs in **remaining_accounts** (required for CPI to succeed)
5. Raydium CPI: transfers 50 USDC in trading fees to **signer_token0/1_account**, and 200 token X to signer's reward ATA
6. lp_handler computes integrator fee: **50 USDC * 12% = 6 USDC** - charged only on trading fees
7. 200 token X ($100) in farming rewards: no fee charged, not converted, not transferred to recipient
8. Fee owner receives 6 USDC but misses ~12 USDC worth of fees on the farming rewards

Scenario 2 - Uncontrolled routing (fee_owner takes all):

1. Same pool and position as above
2. Backend constructs the transaction with fee_owner's reward ATAs in **remaining_accounts** instead of signer's
3. Security layer permits this - fee_owner is whitelisted (security/mod.rs line 402-409)
4. Raydium CPI transfers 200 token X directly to fee_owner's reward ATA
5. Fee owner captures 100% of farming rewards ($100), far exceeding the 12% **fee_percent** cap

RECOMMENDATION

Explicitly handle farming rewards in the post-CPI logic to enforce the same granular control that exists for trading fees:

1. Snapshot farming reward account balances before and after the Raydium CPI
2. Compute integrator fees on farming reward deltas using the same **fee_percent**
3. Enforce that reward recipient accounts belong to the designated **recipient** (or **signer** if **recipient == signer**)
4. Transfer net farming rewards to the recipient's reward token accounts
5. Relax the **NoBalanceChange** check for claim-only calls (**liquidity == 0**) to allow farming-reward-only claims:

RUST

```rust
if liquidity > 0 {
    require_log!(
        signer_token0_amount > 0 || signer_token1_amount > 0,
        LpDepositError::NoBalanceChange
    );
}
```

DEVELOPER RESPONSE

*"**Reward-only** claims were updated so claim-only flows no longer revert due to **NoBalanceChange**, but we intentionally do not charge protocol fees on external farming/incentive rewards. The protocol design charges fees only on the two pool assets, so forgoing fee revenue on external incentive tokens is an intentional product decision."*

5. Enhancement Opportunities

E01: Rename USDC_MIN to USDC_MINT to avoid confusion with a minimum value

LOCATION

solana_lp_handler/17ea3eac/programs/lp_handler/src/state/consts.rs:L33

RELEVANT SNIPPET

>_ consts.rs RUST

```
31: ];
32:
33: pub const USDC_MIN: Pubkey = pubkey!("EPjFWdd5AufqSSqeM2qN1xzybapC8G4wEGGkZwyTDt1v");
34:
35: pub const MIN_USDC_SWAP_AMOUNT: u64 = 1000;
```

DESCRIPTION

The constant **USDC_MIN** holds the USDC mint address but reads as "USDC minimum," which is easy to confuse with the **MIN_USDC_SWAP_AMOUNT** constant defined two lines below (line 35). The intended name is **USDC_MINT**.

POTENTIAL BENEFIT

Eliminates ambiguity between a mint address and a minimum threshold, reducing the risk of misinterpretation during future development or code review.

RECOMMENDATION

Rename **USDC_MIN** to **USDC_MINT** across the codebase.

E02: Inconsistent Token 2022 native mint handling

LOCATION

solana_lp_handler/17ea3eac/programs/lp_handler/src/instructions/zap_common.rs:L826-L835

RELEVANT SNIPPET

>_ zap_common.rs — RUST

```rust
824:  // token account 可能属于 SPL Token 或 Token-2022；按 owner 选择对应 close CPI。
825:  match token_program_2022.as_ref() {
826:      Some(tp22) if token_account.owner == tp22.key => {
827:          token_2022::close_account(CpiContext::new(
828:              tp22.to_account_info(),
829:              token_2022::CloseAccount {
830:                  account: token_account.to_account_info(),
831:                  destination: destination.to_account_info(),
832:                  authority: signer.to_account_info(),
833:              },
834:          ))
835:      }
836:      _ => token::close_account(CpiContext::new(
837:          token_program.to_account_info(),
```

DESCRIPTION

The program checks for the SPL Token 2022 native mint specifically when closing the token account to unwrap SOL. However, in all other parts of the program, only the native mint of the SPL Token is checked, e.g:

zap_common.rs

RUST

```rust
let wsol_mint_key = anchor_spl::token::spl_token::native_mint::ID;
  if accounts.vault_0_mint().key() == wsol_mint_key {
      unwrap_wsol_to_destination(
          user_ai.clone(),
          user_token0_ai.clone(),
          user_ai.clone(),
          token_program_ai.clone(),
          Some(token_program_2022_ai.clone()),
      )?;
  }
  if accounts.vault_1_mint().key() == wsol_mint_key {
      unwrap_wsol_to_destination(
          user_ai.clone(),
          user_token1_ai.clone(),
          user_ai.clone(),
          token_program_ai.clone(),
          Some(token_program_2022_ai),
      )?;
  }
```

decrease_liquidity.rs

```rust
let is_wsol_0 =
    ctx.accounts.vault_0_mint.key() == anchor_spl::token::spl_token::native_mint::ID;
let is_wsol_1 =
    ctx.accounts.vault_1_mint.key() == anchor_spl::token::spl_token::native_mint::ID;
```

Moreover, the SPL Token 2022 native mint is not widely used as of March 2026.

POTENTIAL BENEFIT

Clearly define what native token mint the protocol is intending to support and clean up code.

RECOMMENDATION

Either:

- Introduce a shared helper that recognizes both native SOL mint IDs and use it consistently in all native-SOL special cases.
- Remove the SPL Token 2022 reference in **unwrap_wsol_to_destination**

E03: Removing unused function will improve readability and maintainability

LOCATION

solana_lp_handler/17ea3eac/programs/lp_handler/src/security/mod.rs:L254-L266

RELEVANT SNIPPET

>_ mod.rs RUST

```rust
}

pub fn collect_accounts_to_check<'info>(
    mut ctx_accounts: Vec<AccountInfo<'info>>,
    remaining_accounts: &[AccountInfo<'info>],
) -> Vec<AccountInfo<'info>> {
    // 预留容量避免 extend 时触发二次分配（降低堆内存峰值）。
    // 说明：当前安全层主流程通常只扫描 `ctx.accounts`，此函数主要用于“确实需要把 remaining 合并成一个 Vec”
    // 的场景；如果你担心 OOM，优先使用“分两段循环分别扫描 main + remaining”的方式，避免一次性 Vec 峰值。
    ctx_accounts.reserve(remaining_accounts.len());
    ctx_accounts.extend_from_slice(remaining_accounts);
    // 注意：为降低 SBF 堆内存峰值，这里不再做去重（dedup）。
    // 可能会重复检查同一账户，但能显著减少 Vec 分配与峰值内存，降低 OOM 风险。
    ctx_accounts
}

/// 解析并返回本次指令应使用的安全策略。
```

DESCRIPTION

This function merges **ctx.accounts** and **remaining_accounts** into a single vector, which is exactly what the security layer would need if it wanted to scan all accounts involved in a transaction. However, the helper is never called anywhere in the repo.

Instead, the wrapper passes only **ctx.accounts.to_account_infos()** into the main security checks in lib.rs. As a result, the function is dead code, and its presence is misleading because it suggests the codebase already has a unified "scan all accounts" path when it does not.

POTENTIAL BENEFIT

- Improve code readability and maintainability

RECOMMENDATION

- Remove **collect_accounts_to_check** entirely if the intended design is to keep partial **remaining_accounts** coverage, or
- Use it in the entry and exit security paths if the intent is to validate all reachable accounts

E04: Consistent tick array validation across instructions will improve maintainability and reduce confusion

LOCATION

solana_lp_handler/17ea3eac/programs/lp_handler/src/instructions/swap_and_deposit.rs:L97-L103

RELEVANT SNIPPET

>_ swap_and_deposit.rs — RUST

```rust
pub protocol_position: UncheckedAccount<'info>,

/// CHECK: Raydium CLMM 使用的 TickArray PDA；CPI 时由 Raydium 侧校验/派生
#[account(mut)]
pub tick_array_lower: UncheckedAccount<'info>,

/// CHECK: Raydium CLMM 使用的 TickArray PDA；CPI 时由 Raydium 侧校验/派生
#[account(mut)]
pub tick_array_upper: UncheckedAccount<'info>,

pub memo_program: Program<'info, Memo>,
```

DESCRIPTION

The **swap_and_deposit** instruction declares its tick array accounts as **UncheckedAccount** with no pool constraint:

RUST

```rust
/// CHECK: Raydium CLMM 使用的 TickArray PDA；CPI 时由 Raydium 侧校验/派生
#[account(mut)]
pub tick_array_lower: UncheckedAccount<'info>,

/// CHECK: Raydium CLMM 使用的 TickArray PDA；CPI 时由 Raydium 侧校验/派生
#[account(mut)]
pub tick_array_upper: UncheckedAccount<'info>,
```

The sibling instructions **increase_liquidity** and **decrease_liquidity** apply **AccountLoader** with a **pool_id == pool_state.key()** constraint on the same accounts:

RUST

```rust
#[account(mut, constraint = tick_array_lower.load()?.pool_id == pool_state.key())]
pub tick_array_lower: AccountLoader<'info, TickArrayState>,

#[account(mut, constraint = tick_array_upper.load()?.pool_id == pool_state.key())]
pub tick_array_upper: AccountLoader<'info, TickArrayState>,
```

All three instructions pass these accounts to Raydium CPI, which performs its own internal validation. The Anchor-level constraints in **increase_liquidity** and **decrease_liquidity** serve as defense-in-depth, catching mismatched accounts early with a clear Anchor error rather than a downstream CPI failure. **swap_and_deposit** lacks this layer.

POTENTIAL BENEFIT

Consistent validation across all three instructions reduces the risk of subtle bugs surfacing during future refactors, provides clearer error messages when invalid accounts are passed (Anchor constraint error vs opaque CPI failure), and saves compute units by failing early before executing the swap CPI.

RECOMMENDATION

Unify the tick array account types across all three instructions. Since Raydium CPI validates these accounts regardless, either approach is functionally correct.

E05: Enforcing non-empty fee_owners whitelist will reduce risk of admin error

LOCATION

solana_lp_handler/17ea3eac/programs/lp_handler/src/instructions/security_config.rs:L42-L50

RELEVANT SNIPPET

>_ security_config.rs RUST

```rust
    fee_owners: Vec<Pubkey>,
) -> Result<()> {
    require!(!pools.is_empty(), LpDepositError::SecurityConfigPoolsEmpty);
    require!(
        pools.len() <= MAX_ALLOWED_POOLS,
        LpDepositError::MathOverflow
    );
    require!(
        fee_owners.len() <= MAX_FEE_OWNERS,
        LpDepositError::MathOverflow
    );
    let cfg = &mut ctx.accounts.security_config;
    cfg.pools = pools.clone();
```

DESCRIPTION

init_security_config and **update_security_config** allow **fee_owners = []** because they only enforce an upper bound on the vector length. However, the runtime security layer treats an empty fee-owner whitelist as invalid and requires both **fee_owners_n != 0** and membership of the provided **fee_owner** before any protected instruction can execute.

As a result, a config state that is considered valid by the write path is considered invalid by the read path. Once **fee_owners** is set to an empty list, every instruction that passes through the security wrapper reverts with **InvalidFeeOwner**, since no account can satisfy membership in an empty whitelist.

POTENTIAL BENEFIT

Prevent admin error and unexpected protocol DOS.

RECOMMENDATION

Enforce **!fee_owners.is_empty()** in both **init_security_config** and **update_security_config** so the config writer and runtime security checks share the same invariant.

E06: Validating tick parameters against existing position in increase_liquidity will improve deposit efficiency by preventing unnecessary swap fees

LOCATION

solana_lp_handler/17ea3eac/programs/lp_handler/src/instructions/increase_liquidity.rs:L183-L184

RELEVANT SNIPPET

>_ increase_liquidity.rs RUST

```rust
amount_1_in: u64,
return_mint: Option<Pubkey>,
tick_lower_index: i32,
tick_upper_index: i32,
slippage_bps: u16, // 滑点，单位为基点 (1 bps = 0.01%)
swap_amount_in: u64,
```

DESCRIPTION

The **increase_liquidity** instruction accepts **tick_lower_index** and **tick_upper_index** as caller-supplied parameters and passes them to **prepare_zap_plan_and_swap_if_needed**. These control out-of-range detection, auto-swap direction/amount, and liquidity computation. However, there is no on-chain validation that these values match **personal_position.tick_lower_index** and **personal_position.tick_upper_index** - the stored ticks of the position being increased.

Raydium's **increase_liquidity_v2** ignores the caller's tick parameters entirely, reading them from **PersonalPositionState**. This creates a mismatch: lp_handler's pre-swap logic uses potentially incorrect tick values, while Raydium's CPI uses the correct stored values.

When mismatched ticks cause lp_handler to incorrectly identify the price as out-of-range, it triggers the auto-swap override (lines 192-222), converting the user's entire balance of one token to the other. Since the position's actual range differs, the Raydium CPI receives a mismatched token ratio, adding minimal liquidity. The leftover is swapped back, costing the user two rounds of swap fees for no useful outcome.

For **swap_and_deposit**, the tick parameters serve a legitimate purpose (defining the new position's range). But for **increase_liquidity**, the ticks should be read from the existing **personal_position** account.

POTENTIAL BENEFIT

Prevents users from incurring unnecessary double swap fees due to mismatched tick parameters. For a $10,000 deposit in a 0.25% fee pool, the worst case is ~$50 in wasted swap fees (2 x 0.25% x $10,000). Also improves robustness against frontend bugs that could pass incorrect tick values.

RECOMMENDATION

Validate that the caller-supplied tick parameters match the existing position's stored ticks:

RUST

```
require!(
    tick_lower_index == ctx.accounts.personal_position.tick_lower_index,
    LpDepositError::InvalidTickRange
);
require!(
    tick_upper_index == ctx.accounts.personal_position.tick_upper_index,
    LpDepositError::InvalidTickRange
);
```

Alternatively, read the ticks directly from **personal_position** instead of accepting them as instruction parameters, since the position's range is immutable once created.

DEVELOPER RESPONSE

*"**increase_liquidity** now enforces that the supplied **tick_lower_index** and **tick_upper_index** match the existing **personal_position**."*

E07: Renaming convert_to_usdc to convert_to_target_mint in decrease_liquidity will improve clarity

LOCATION

solana_lp_handler/17ea3eac/programs/lp_handler/src/instructions/decrease_liquidity.rs:L27

RELEVANT SNIPPET

>_ decrease_liquidity.rs RUST

```rust
  slippage_bps: u16, // 滑点，单位为基点 (1 bps = 0.01%)
  fee_percent: u16,
  convert_to_usdc: bool,
)]
pub struct DecreaseLiquidity<'info> {
```

DESCRIPTION

The **convert_to_usdc: bool** parameter name is misleading. In the implementation, this flag does not enforce USDC conversion; it enables conversion into whichever mint is passed via **swap_to_token_mint** (**token0** or **token1**). This naming mismatch can confuse integrators, reviewers, and client SDK users, and may cause incorrect assumptions in routing and settlement logic.

POTENTIAL BENEFIT

1. Clearer API semantics and lower integration error risk.
2. Easier auditing and reasoning about settlement behavior.
3. Better long-term maintainability.

RECOMMENDATION

Rename **convert_to_usdc** to **convert_to_target_mint**.

About Us

About Adevar Labs

Adevar Labs is a boutique blockchain security firm specializing in web3 audits.

Built by a mix of experienced professionals in traditional enterprise and crypto natives who have contributed to some of the most critical projects in blockchain infrastructure.

Our team's background spans companies like Bitdefender, Asymmetric Research, Quantstamp, Chainproof, and Juicebox, and includes experience securing smart contracts, bridges, and L1 and L2 protocols across ecosystems like Solana, Ethereum, Polkadot, Cosmos, and MultiversX.

With over 100 audits completed and a portfolio that includes custom fuzzers, exploit modeling, and runtime testing frameworks, Adevar Labs brings both depth and precision to every engagement.

Our auditors have discovered critical vulnerabilities, built high-impact tooling, and placed in top positions in premier audit competitions including Code4rena and Sherlock.

Team members hold distinctions such as PhDs in software protection, and key roles at Fortune 500 companies, and leadership of flagship conferences like ETH Bucharest.

With team members having publications with over 1,100 academic citations and trusted by projects securing over $500M in on-chain value, Adevar Labs blends elite technical rigor with real-world security impact.

We also collaborate with some of the best independent security researchers in the web3 space.

Projects may optionally request specific contributors to be part of their audit.

Audit Methodology

Our audit methodology is specialized to provide thorough security assessments of Solana programs. We focus explicitly on rigorous manual analysis, detailed threat modeling, and careful validation of implemented fixes to ensure your programs operate securely and as intended.

1. Program Context and Architecture Analysis

Our auditors begin by deeply examining your Solana program's documentation, intended functionality, and account design. We meticulously map out program interactions, instruction processing flows, and state management logic. Special attention is given to understanding how your program interfaces with critical Solana system programs, such as the SPL Token Program, Stake Program, and System Program. Last but not least we check external integrations with other Solana projects and verify if the inputs and return values are handled properly.

2. Threat Modeling

We conduct targeted threat modeling tailored specifically for Solana's execution environment. We carefully define attacker capabilities and identify potential vulnerabilities that may arise from Solana-specific issues, including but not limited to:

- Unauthorized account data manipulation
- Improper ownership or signer verification
- Misuse of Program-Derived Addresses (PDAs)
- Incorrect use of Cross-Program Invocations (CPI)
- Failure to adequately handle account privileges or account states
- Risks stemming from rent-exemption and account initialization logic

3. In-depth Manual Security Review

Our experienced auditors perform an extensive manual security review of your Rust-based Solana programs. This involves a comprehensive line-by-line inspection of source code, focusing on common Solana vulnerabilities including, but not limited to:

- Missing or insufficient ownership checks
- Inadequate signer checks
- Incorrect handling of CPI calls and invocation privileges
- Arithmetic and integer overflow or underflow errors
- Unsafe deserialization and serialization of account data structures
- Improper token transfers and SPL-token logic issues
- Logic flaws in financial operations or state transitions
- Edge-case handling in instruction input validation
- Potential denial-of-service vectors related to transaction execution and account handling

During this stage, we clearly document any discovered vulnerabilities, including detailed descriptions, precise severity ratings, and recommendations for secure implementation. In situations where it is not clear how the vulnerability might be exploited we may also include a detailed proof-of-concept exploit including code snippets and instructions on how the exploit could be performed.

4. Detailed Fix Review and Validation

After the initial audit and your team's subsequent remediation efforts, we perform a comprehensive fix review to ensure the vulnerabilities identified have been effectively resolved.

We verify each fix individually, confirming that:

- Corrections effectively eliminate the security risks
- Changes do not inadvertently introduce new vulnerabilities or regressions
- The fixes align closely with Solana best practices and secure coding guidelines

Our detailed validation ensures that security improvements are robust, complete, and aligned with best practices specific to the Solana development ecosystem.

Confidentiality Notice

This report, including its content, data, and underlying methodologies, is subject to the confidentiality and feedback provisions in your agreement with Adevar Labs. These materials are not to be disclosed, extracted, copied, or distributed except to the extent expressly authorized by Adevar Labs.

Legal Disclaimer

The review and this report are provided by Adevar Labs on an as-is, where-is, and as-available basis. To the fullest extent permitted by law, Adevar Labs disclaims all warranties, expressed or implied, in connection with this report, its content, and your use thereof, including, without limitation, the implied warranties of merchantability, fitness for a particular purpose, and non-infringement.

You agree that access to and/or use of the report and other results of the review, including any associated services, products, protocols, platforms, content, and materials, will be at your sole risk.

FOR AVOIDANCE OF DOUBT, THE REPORT, ITS CONTENT, ACCESS, AND/OR USAGE THEREOF, INCLUDING ANY ASSOCIATED SERVICES OR MATERIALS, SHALL NOT BE CONSIDERED OR RELIED UPON AS ANY FORM OF FINANCIAL, INVESTMENT, TAX, LEGAL, REGULATORY, OR OTHER ADVICE. This report is based on the scope of materials and documentation provided for a limited review at the time provided.

You acknowledge that blockchain technology remains under development and is subject to unknown risks and flaws. Adevar Labs does not warrant, endorse, guarantee, or assume responsibility for any product or service advertised or offered by a third party, or any open-source or third-party software, code, libraries, materials, or information accessible through the report. As with the purchase or use of a product or service in any environment, you should use your best judgment and exercise caution where appropriate.